- Fivemat[15]
- Fuubar[16]
- NyanCat[17]
- Any others you can find

Install a couple of these gems onto your system. Try each formatter at the command line with one of your projects; the expense tracker app you wrote in the second part of this book would be perfect.

Once you've found a formatter you like, configure one of your projects to use it on every run. If you *really* enjoy using this formatter, you may want to configure RSpec to use it on all your projects.

Detecting Slow-Running Specs

As we've discussed before, keeping your specs running quickly is key to achieving a productive flow. In this exercise, you're going to write a library that will measure some of your specs' execution times and fail any example that's too slow.

You don't want to apply the same timing requirements to *all* your specs—after all, integration and acceptance specs are typically slower than unit specs. Your library should use a piece of configurable metadata, such as :fail_if_slower_than, to set the threshold. For instance, any example in the following group should fail if it takes longer than a hundredth of a second to run:

09-configuring-rspec/exercises/fail_if_slower_than_spec.rb

```
RSpec.describe SomeFastUnitSpecs, fail_if_slower_than: 0.01 do
  # ...
end
```

Users will be able to configure these thresholds automatically for whole sections of their suites, using the techniques in *Derived Metadata*, on page 135.

When your library first loads, it should define an around configuration hook. The hook will compare the wall clock time before and after each example runs, and then fail the example if it takes too long.

15. https://github.com/tpope/fivemat
16. https://github.com/thekompanee/fuubar
17. https://github.com/mattsears/nyan-cat-formatter